

RAILEASE

TRAIN RESERVATION SYSTEM

PROJECT REPORT

A Core Java project that demonstrates the basic process of finding a train, booking seats, keeping reservation details and cancelling a ticket.

Project	RailEase — Train Reservation System
Language	Java
Application Type	Console-based application
Main Concepts	OOP, ArrayList, Exception Handling, LocalDate, UUID
Storage	In-memory data during program execution
Version Control	GitHub
Prepared By	Shreya
Branch / Course	CSE

NAME: S Shreya

REGISTRATION NUMBER: 25BAI11620

ACADEMIC YEAR: 2026

PROJECT TYPE: Core Java/Object-Oriented Programming Project

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References
15. Conclusion

Introduction

RailEase is a simple Train Reservation System made using Core Java. I chose this topic because railway reservation is a process that most people are familiar with, and it gives a good example of how different Java concepts can work together in one project.

The application is menu-driven, so the user can select an operation from the console. The main operations are customer registration, viewing trains, searching for a train, booking a ticket, viewing reservations and cancelling a ticket.

The current version keeps the data in Java objects and ArrayLists while the program is running. A database is not necessary for this version, which keeps the project focused on Core Java.

Main idea

Customer → Find Train → Check Seats → Book Ticket → Generate Transaction ID

↓

View / Cancel Booking

Problem Statement

Railway booking involves more than just selecting a train. The user has to know which trains are available, check the number of seats, calculate the fare and keep track of the booking. If these steps are done manually, it is easier to make mistakes or lose track of information.

The problem taken in this project is to build a small software system that combines these basic activities in one place. The system should make booking straightforward and should also update the seat count when a ticket is booked or cancelled.

The project therefore focuses on three things: keeping information organized, applying the correct booking logic, and giving clear feedback to the user.

Problems addressed

- Finding train information quickly.
- Checking whether enough seats are available.
- Calculating the ticket amount automatically.
- Keeping reservation details together.
- Cancelling a ticket and restoring its seats.

Functional Requirements

ID	Function	What the system does
FR-01	Register customer	Accepts and stores customer details.
FR-02	View trains	Shows train number, route, seats and fare.
FR-03	Search train	Finds a train using its train number.
FR-04	Book ticket	Checks details and creates a reservation.
FR-05	Calculate fare	Uses seats × fare per seat.
FR-06	Generate ID	Creates a unique transaction ID.
FR-07	View reservations	Displays existing bookings.
FR-08	Cancel ticket	Cancels a booking and restores seats.
FR-09	Validate input	Handles invalid values and missing records.
FR-10	Exit	Closes the application.

Non-Functional Requirements

Usability: The menu and messages should be easy for a first-time user to understand.

Reliability: The program should not change seat or reservation data when a booking is invalid.

Performance: For the small set of trains used here, operations should be fast enough for normal console use.

Maintainability: Different responsibilities should be kept in different classes.

Portability: The application should run wherever a compatible Java JDK is installed.

Readability: The code and output should be simple enough for another student to follow.

Extensibility: The design should allow a database or GUI to be added later.

Current limitations

- Data is lost when the program stops.
- There is no real payment gateway.
- There is no live railway API.
- The current interface is console based.
- Security is kept basic because this is an academic project.

System Architecture

I divided the project into a few small classes instead of putting everything inside Main.java. This makes the program easier to understand and makes it easier to change one part without disturbing the others.

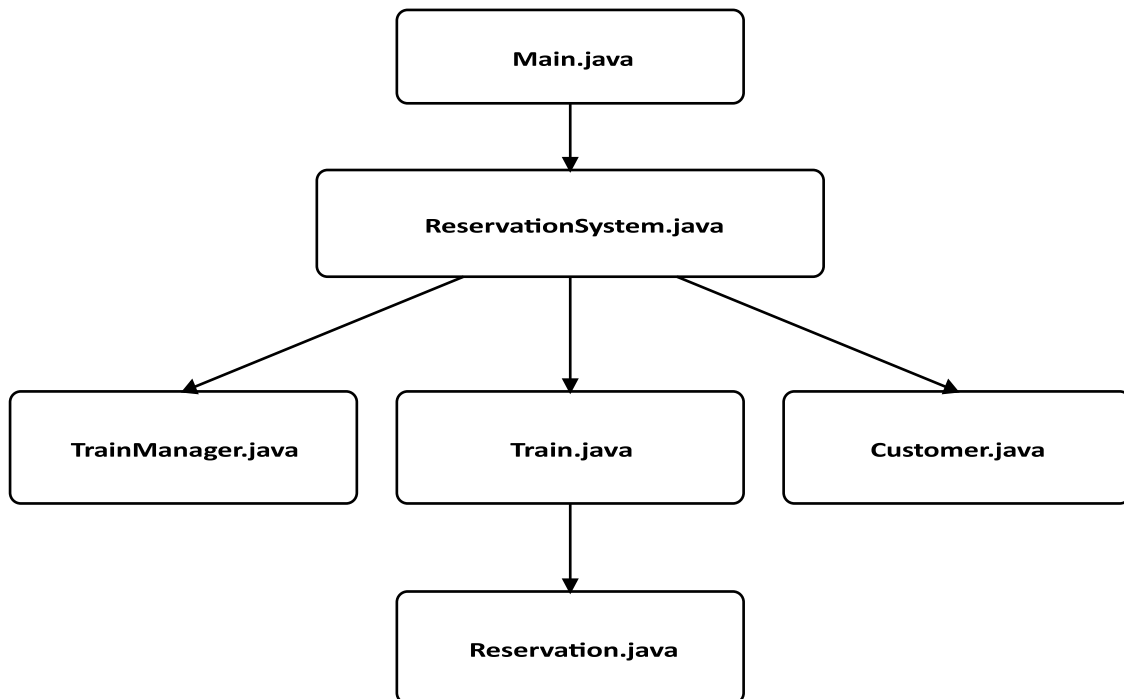


Figure 1 — Basic class-level architecture used in the project.

Class	Main responsibility
Main.java	Starts the application.
ReservationSystem.java	Controls the menu and main user operations.
TrainManager.java	Stores and searches the train list.
Train.java	Stores train details and manages seat count.
Customer.java	Stores customer details.
Reservation.java	Stores booking details and total fare.

Design Diagrams

Use Case Diagram

The main actor in the current version is the customer. The customer interacts with the system for registration, train search, booking, reservation viewing and cancellation.

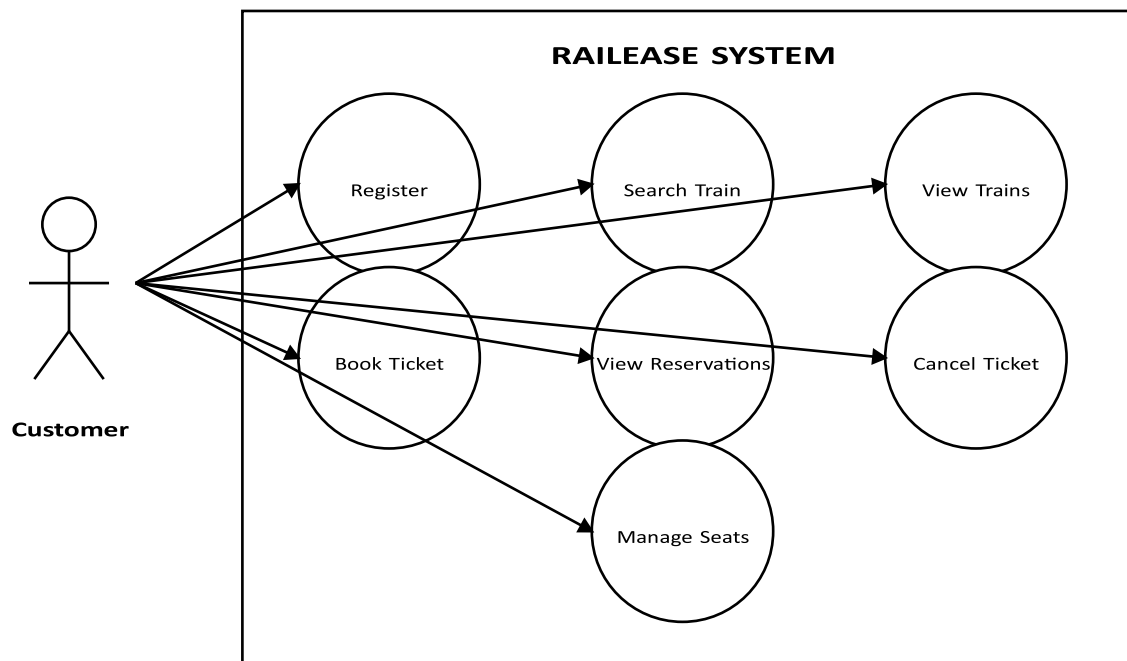


Figure 2 — Use cases of the current system.

Workflow Diagram

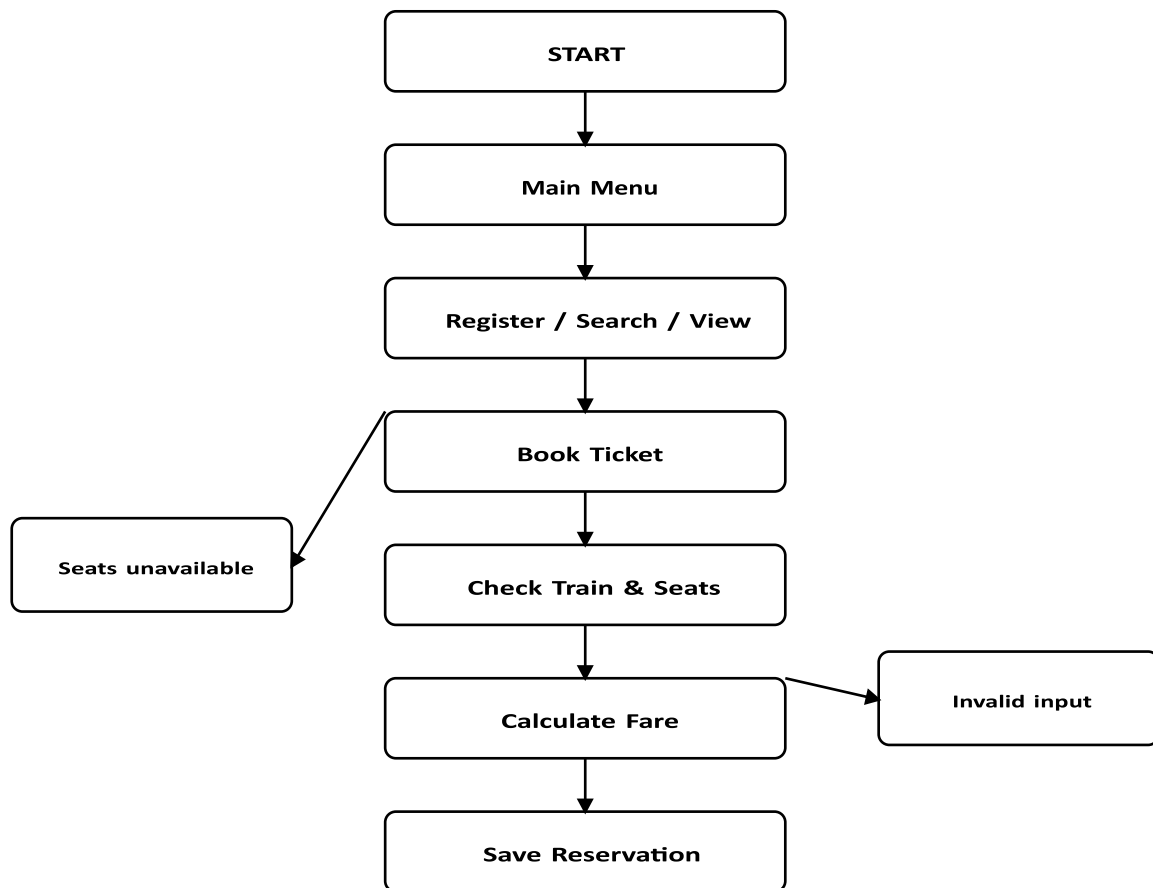


Figure 3 — Overall booking workflow.

Sequence Diagram

The sequence below shows what happens when a customer books a ticket. The system first receives the details, checks the train and seats, then creates the reservation.

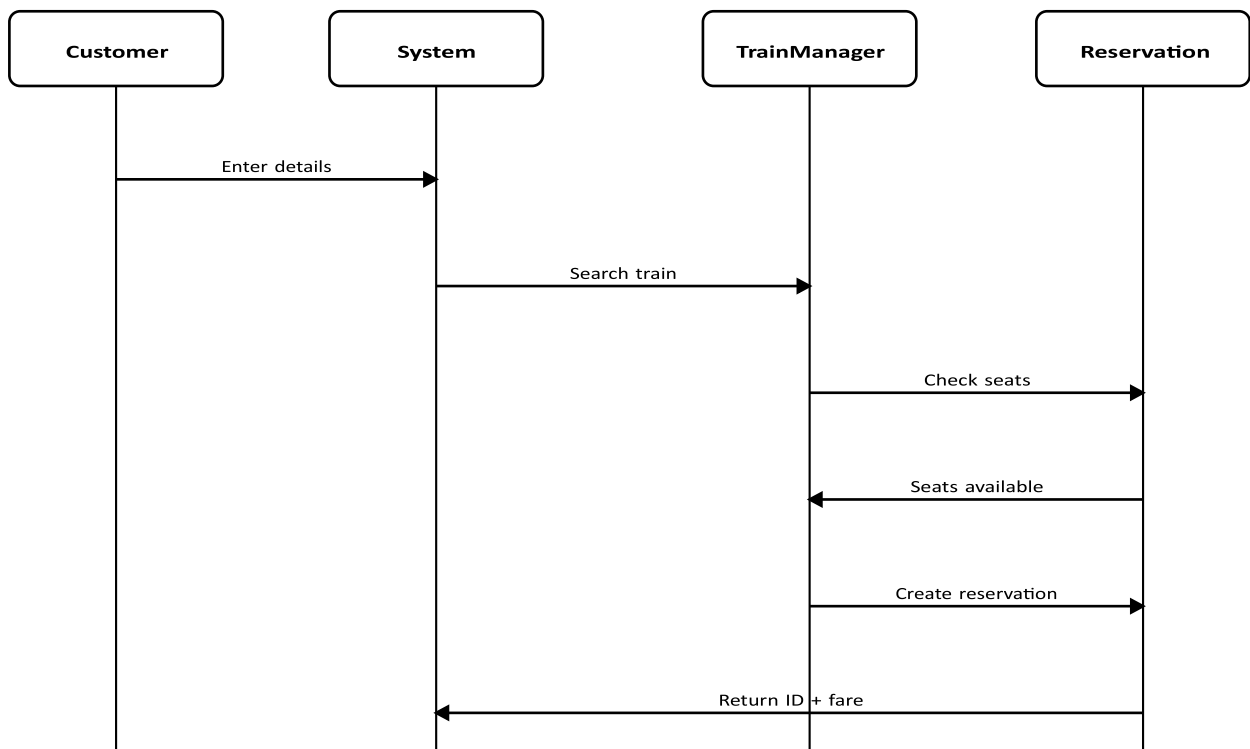


Figure 4 — Ticket booking sequence.

Class / Component Diagram

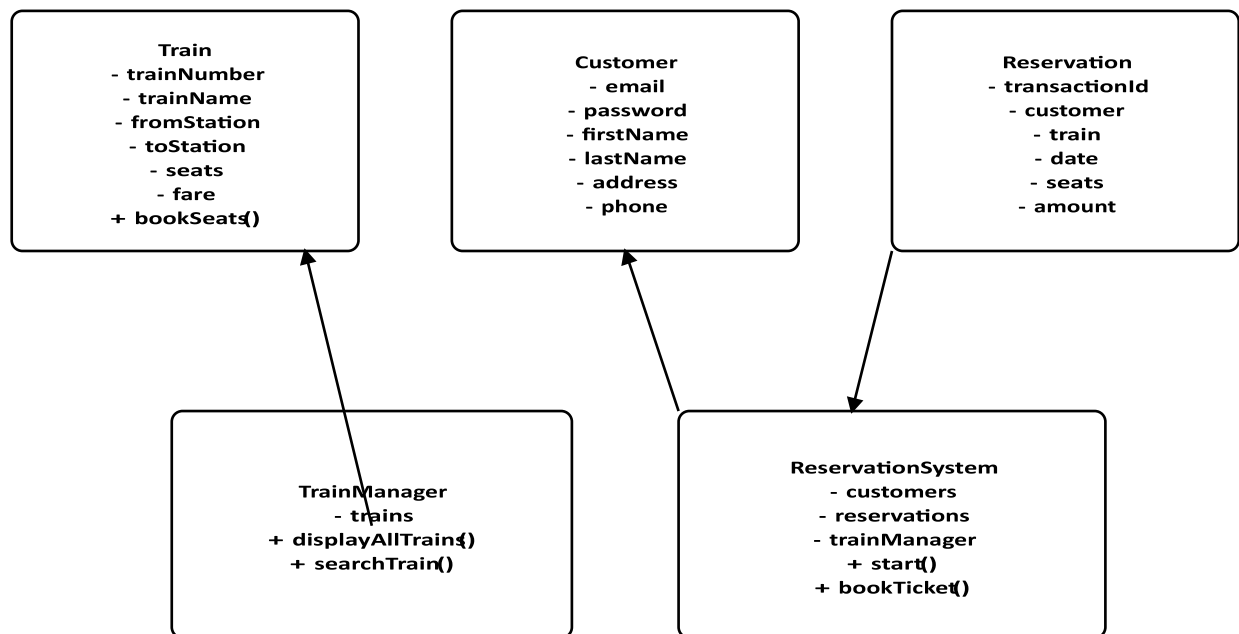


Figure 5 — Main classes and their responsibilities.

ER Diagram (for future storage)

The current program uses ArrayList, so it does not need a database. I have still included a simple ER model because the same project can later be connected to Oracle or MySQL using JDBC.

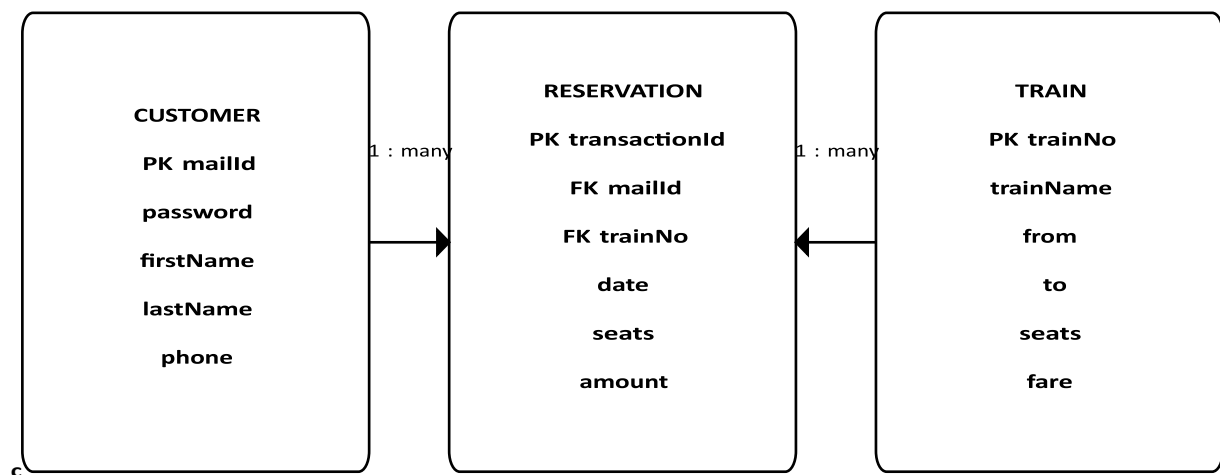


Figure 6 — Conceptual database model for a future JDBC version.

Design Decisions & Rationale

Separate classes: I used separate classes because Train, Customer and Reservation represent different real-world objects.

ArrayList: ArrayList is enough for the current small, in-memory dataset and is easy to work with in Core Java.

Encapsulation: Fields are kept private so that the object controls how its data is accessed or changed.

LocalDate: Using LocalDate is clearer than storing a journey date as an ordinary string.

UUID: A generated UUID gives each reservation a different transaction identifier without manually maintaining a counter.

Train controls seats: The Train class changes its own seat count, which keeps seat-related logic in one place.

ReservationSystem as controller: The menu and user-flow logic stay separate from the data classes.

In simple terms, the design follows the idea of giving each class one main job.

Implementation Details

The application is implemented using Core Java. The program starts from Main.java and then uses the ReservationSystem class to show the menu and handle the user's choices.

Important implementation points

- Train: contains train number, name, source, destination, seats and fare.
- Customer: contains email, password, name, address and phone number.
- Reservation: connects a customer and a train with a date, seat count and amount.
- TrainManager: maintains the train list and provides search/display operations.
- ReservationSystem: handles registration, booking, viewing and cancellation.

Seat booking logic

```
if (numberOfSeats <= seats) { seats -= numberOfSeats; return true;
}
return false;
```

Fare calculation

```
double amount = train.getFare() * numberOfSeats;
```

Transaction ID generation

```
String transactionId = UUID.randomUUID().toString();
```

Journey date

```
LocalDate date = LocalDate.parse(dateInput);
```

Sample train data

Train No.	Name	From	To	Seats	Fare
10001	JODHPUR EXP	HOWRAH	JODHPUR	152	490.50
10002	YAMUNA EXP	GAYA	DELHI	52	550.50
10003	NILANCHAL EXP	GAYA	HOWRAH	92	451.00
10004	JAN SATABDI EXP	RANCHI	PATNA	182	550.00
10005	GANGE EXP	MUMBAI	KERALA	12	945.00
10006	GARIB RATH EXP	PATNA	DELHI	1	1450.75

1. Screenshots / Results

These screenshots show the main stages of the application. They are included to show that the project has been tested through the actual console workflow.



Figure 7 — Main menu — the user can choose the required operation.

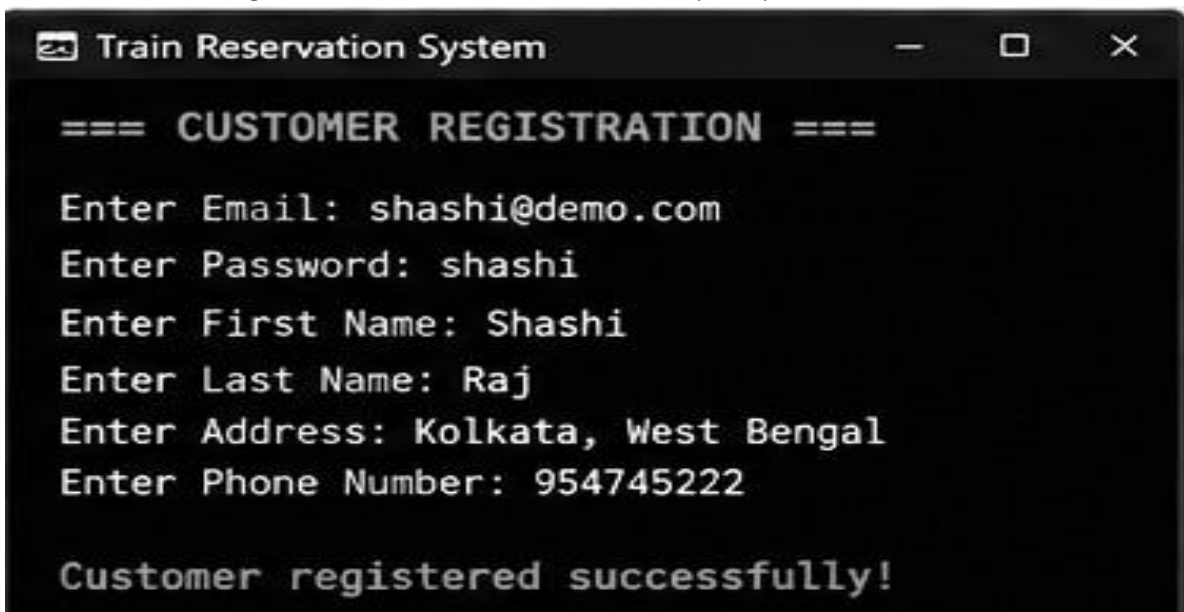


Figure 8 — Customer registration — entering customer details.

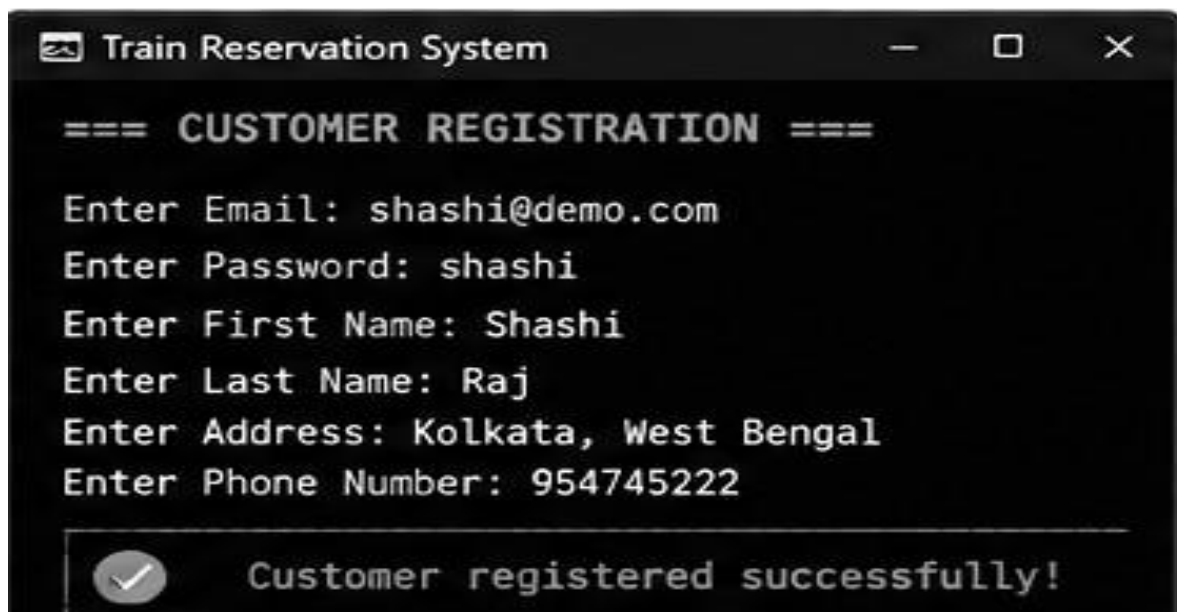


Figure 9 — Registration success — confirmation after valid registration.

The screenshot shows the same "Train Reservation System" window, now displaying the "ALL TRAINS" section. It lists six trains with their respective details in a table format.

No.	Train Name	From	To	Seats	Fare
10001	JODHPUR EXP	HOWRAH	JODHPUR	152	₹490.50
10002	YAMUNA EXP	GAYA	DELHI	52	₹550.50
10003	NILANCHAL EXP	GAYA	HOWRAH	92	₹451.00
10004	JAN SATABDI EXP	RANCHI	PATNA	182	₹550.00
10005	GANGE EXP	MUMBAI	KERALA	12	₹945.00
10006	GARIB RATH EXP	PATNA	DELHI	1	₹1450.75

Figure 10 — Available trains — displaying train information.

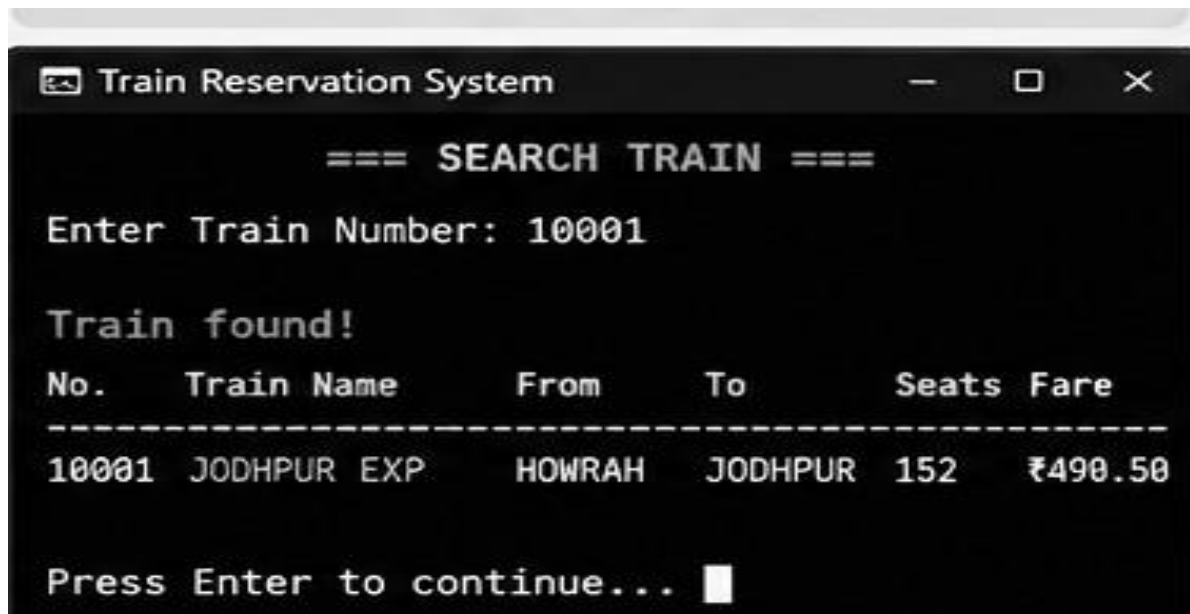


Figure 11 — Train search — displaying the selected train.

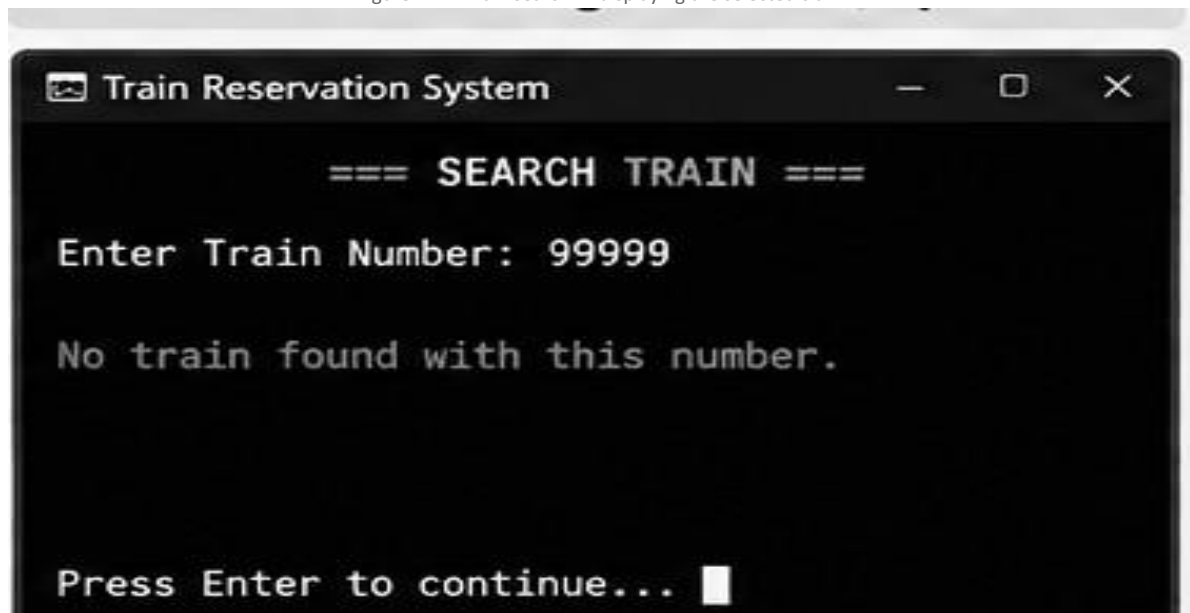


Figure 12 — Train search result — handling an unsuccessful search.

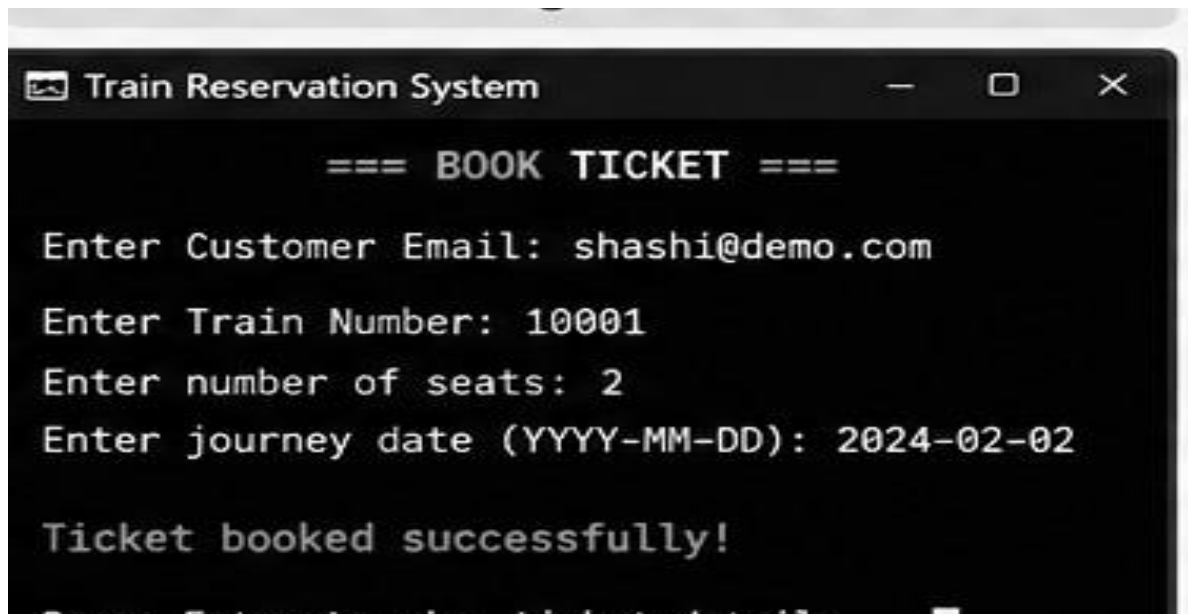


Figure 13 — Book ticket — entering booking details.

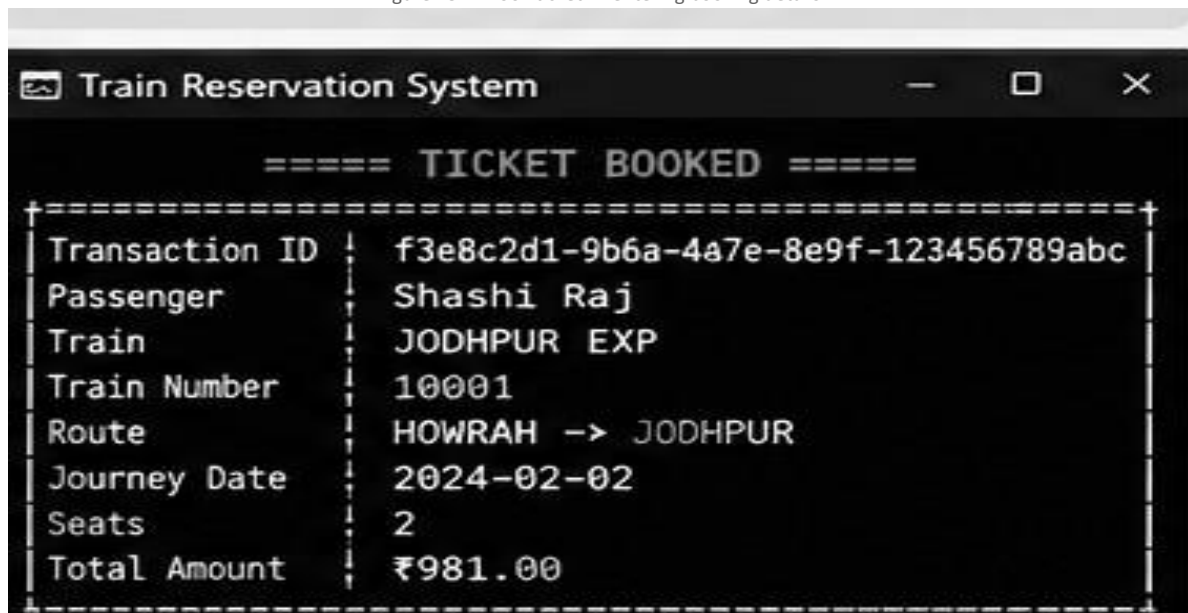


Figure 14 — Booking success — showing the completed reservation.

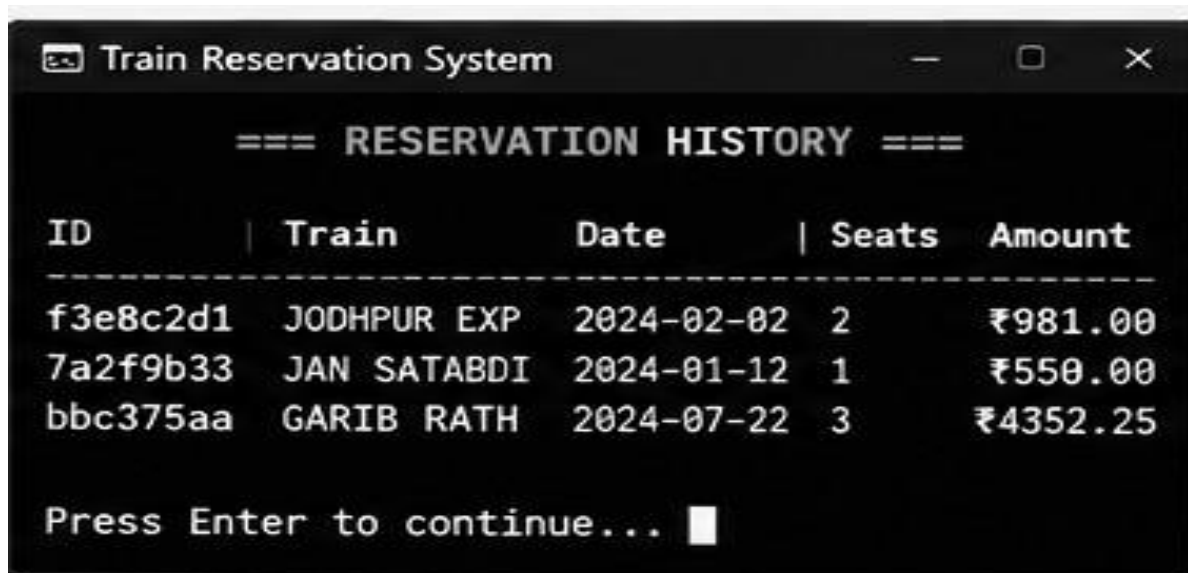


Figure 15 — Reservation history — viewing existing bookings.

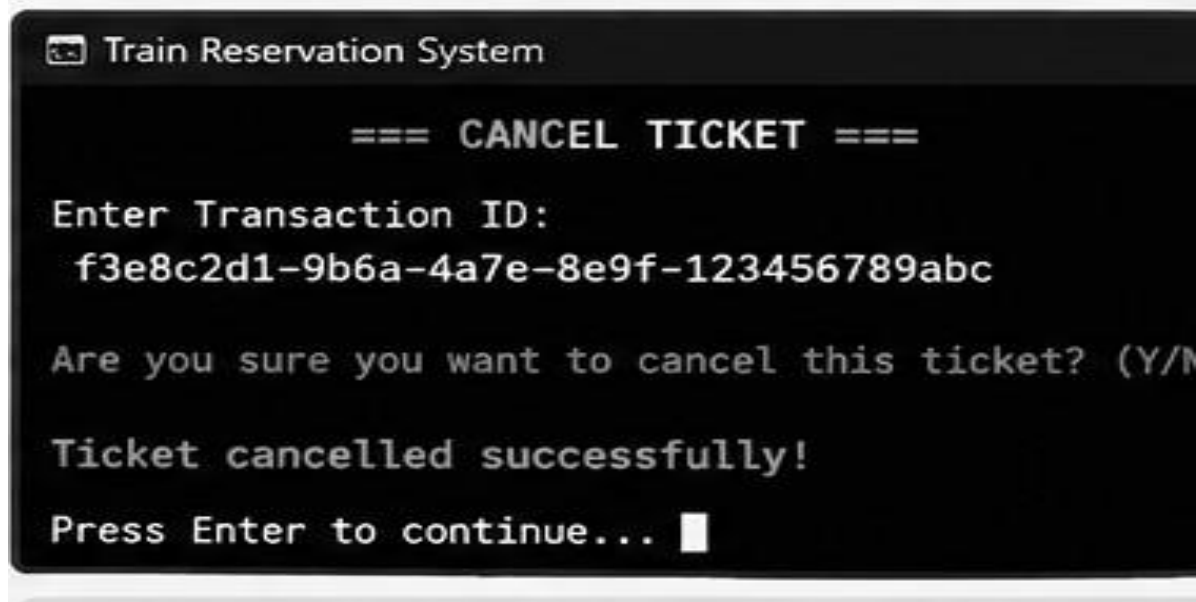


Figure 16 — Ticket cancellation — cancelling a reservation.

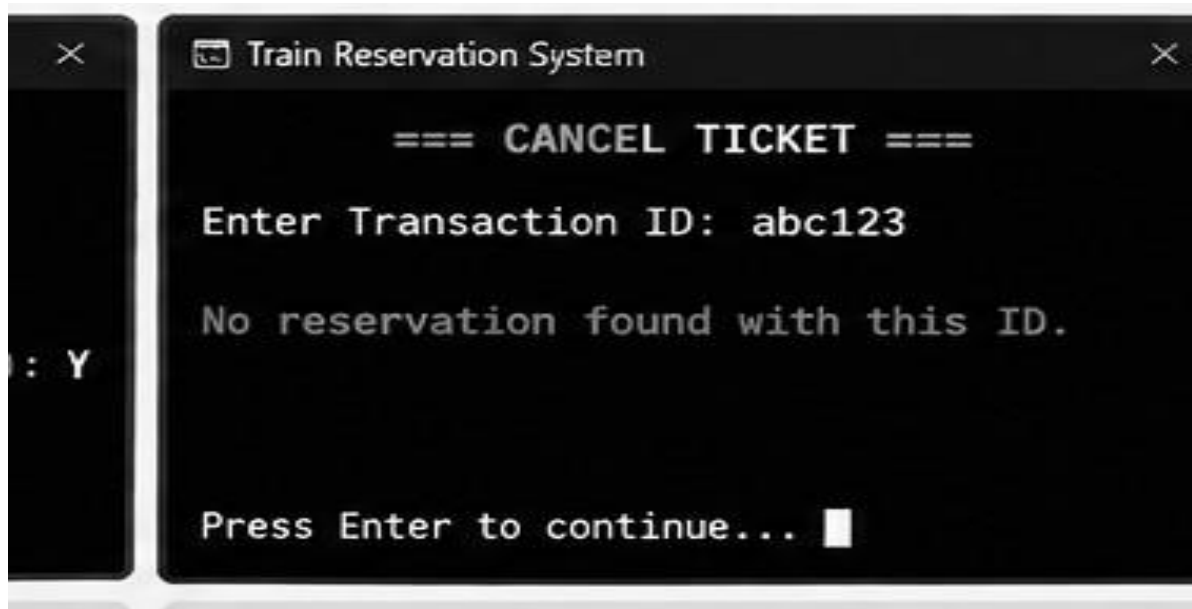
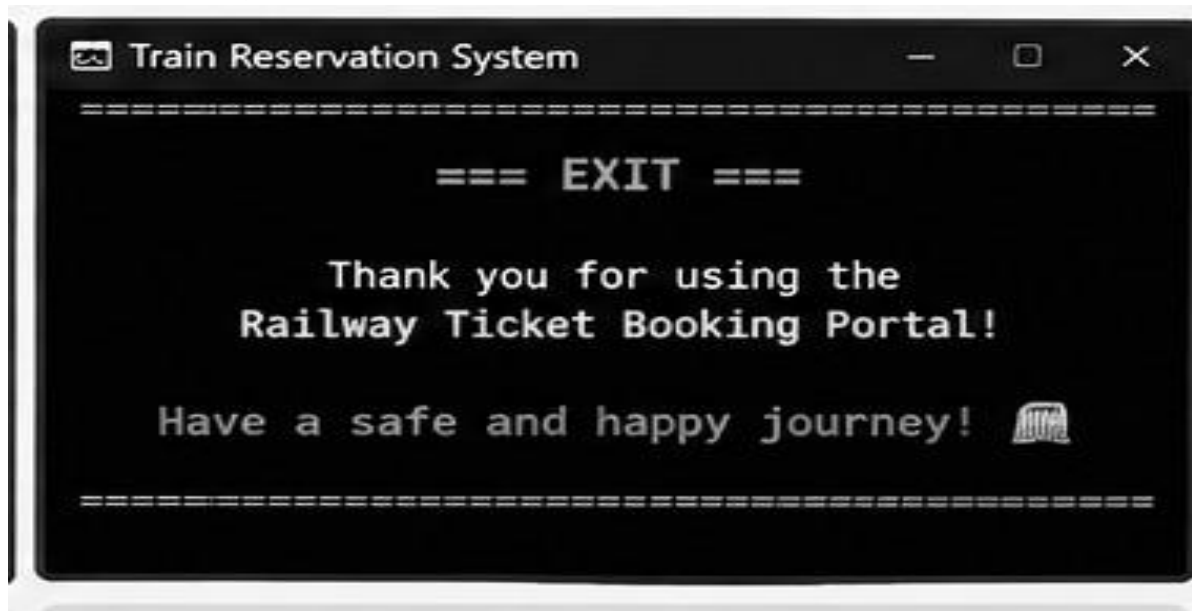


Figure 17 — Invalid cancellation — handling an unknown transaction ID.



2. Testing Approach

I tested the application by trying both normal inputs and inputs that should be rejected. The main purpose was to check that the reservation state changes only when the operation is valid.

ID	Test	Input	Expected Result
TC-01	Register customer	Valid details	Registration succeeds
TC-02	View trains	Option 2	Train list appears
TC-03	Search train	10001	Correct train displayed
TC-04	Search train	99999	Not found message
TC-05	Book ticket	Valid train + seats	Booking succeeds
TC-06	Book ticket	Too many seats	Booking rejected
TC-07	Seat validation	0 / negative	Invalid input message
TC-08	Date validation	Wrong date format	Date error handled
TC-09	View reservations	Existing booking	Reservation displayed
TC-10	Cancel ticket	Valid transaction ID	Ticket cancelled
TC-11	Cancel ticket	Wrong transaction ID	Not found message
TC-12	Exit	Exit option	Program closes

Testing summary

The tests cover the main successful flow as well as common invalid cases. In particular, seat availability, invalid input and cancellation were checked because these operations directly affect reservation data.

3. Challenges Faced

Keeping seat counts correct: I had to make sure a successful booking reduced the seats and a cancellation added back the same number.

Handling Scanner input: Mixing numbers and strings can cause skipped inputs in Java, so input handling had to be done carefully.

Date input: A user can enter a date in the wrong format, so LocalDate parsing needs exception handling.

Finding a reservation: Cancellation depends on the transaction ID, so the program must search the reservation list correctly.

Avoiding one large class: It was tempting to put everything in one file. Splitting the project into classes made the code easier to manage.

Preparing for future changes: Even though the current version does not use a database, the classes were kept separate so JDBC can be added later.

These were useful challenges because they helped me understand that writing a working program also involves handling unexpected input and keeping the program organized.

4. Learnings & Key Takeaways

Working on this project helped me understand how basic Java concepts are used in an actual application instead of only in small individual programs.

- I learned how to divide a real-world problem into classes and objects.
- I got more comfortable with ArrayList and storing multiple objects.
- I understood why encapsulation is useful when data should not be changed directly.
- I practiced exception handling for invalid dates and inputs.
- I learned how to update object state when a booking or cancellation happens.
- I understood how a transaction ID can be generated and used later.
- I learned that a simple project can still benefit from proper structure and documentation.
- I also learned how screenshots and test cases can be used as evidence that the program was checked.

Main takeaway

For me, the biggest takeaway is that programming is not only about writing code that runs. It is also about making the code understandable, handling wrong inputs and designing it so that it can be improved later.

5. Future Enhancements

Database integration: Use JDBC with Oracle or MySQL so that customers, trains and bookings are stored permanently.

Login system: Add separate customer and admin login with proper password security.

Admin module: Allow an administrator to add trains, remove trains and update fares or seat capacity.

GUI: Create a graphical version using Java Swing or JavaFX.

PNR and passenger details: Generate PNR numbers and allow multiple passenger details in one booking.

Waiting list: Add waiting-list functionality when no seats are available.

Refund handling: Calculate and store refund information after cancellation.

Notifications: Send booking and cancellation confirmations through email or other notification methods.

Planned growth

```
graph TD; A[Current version] --> B[Core Java + ArrayList + Console]; B --> C[JDBC + Database]; C --> D[Login + Admin Module]; D --> E[GUI + Reports + Notifications];
```

6. References

- Oracle Java Documentation — Java SE API documentation.
- Oracle Java Tutorials — Core Java and Object-Oriented Programming concepts.
- Eclipse IDE Documentation — Java project creation and execution.
- Git Documentation — version control basics.
- GitHub Documentation — repositories and Markdown README files.
- The Java source files and console outputs created for this project.
- The academic report requirements provided for this project.

7. Conclusion

RailEase is a small but complete example of a railway reservation workflow implemented using Core Java. It covers customer registration, train search, seat checking, ticket booking, fare calculation, reservation history and cancellation.

The project helped me connect Java concepts such as classes, objects, ArrayList, methods, exception handling, LocalDate and UUID with a practical problem.

The current version is intentionally simple and works through the console. However, the class structure gives a clear starting point for adding a database, login system, admin functions and a graphical interface in the future.

Overall, the project shows how a real-world process can be broken into smaller programming tasks and then combined into one working application.

— End of Report —